

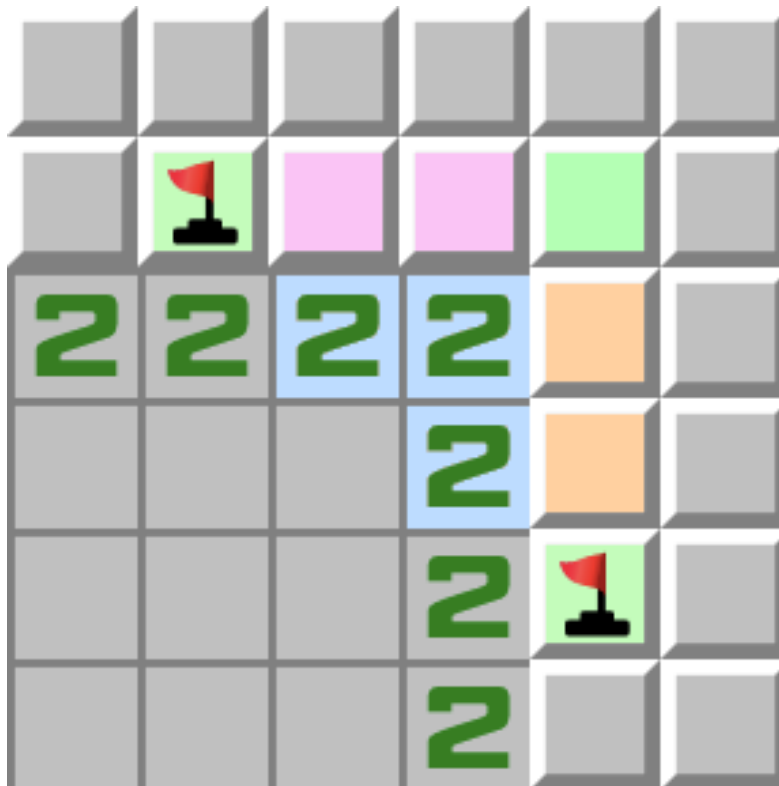
Software Requirements Specification (SRS)

Project: MiniSweeper

Course: הנדסת תוכנה ואבטחת מידע

ElephantsTeam

Students: Asil Alshikh - 214160707, Weaam Shehade - 213777071, Juna Khateeb - 325469575, Reem Ali - 324992940



1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the requirements for the MiniSweeper application. It establishes a mutual understanding between developers, evaluators, and stakeholders regarding system behavior, scope, and quality attributes. This document will serve as a reference during design, implementation, testing, and maintenance.

1.2 Scope

MiniSweeper is a two-player desktop game developed in Java. It follows the Model-View-Controller (MVC) design pattern and allows players to uncover cells, avoid mines, and answer quiz questions. The system includes difficulty levels, score and life tracking, and data storage using CSV files. The software runs offline and provides a graphical user interface (UI).

1.3 Definitions, Acronyms, and Abbreviations

SRS – Software Requirements Specification

MVC – Model View Controller

CSV – Comma-Separated Values

UI – User Interface

FR – Functional Requirement

1.4 References

IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.

Practice 2 – Scrum & SRS (2025).

1.5 Overview

Section 2 presents the system overview, Section 3 defines all Functional Requirements (FR) by features, Section 4 discusses non-functional requirements, and Section 5 concludes with appendices and template justification.

2. Overall Description

2.1 Product Perspective

MiniSweeper is a standalone Java application using MVC. The Model handles logic and data (players, board, cells), the View provides the UI, and the Controller mediates user interaction. No external servers or databases are used; data is stored locally via CSV files.

2.2 Product Functions

Main functions include initializing a new game, managing turns, interacting with cells, answering quiz questions, and storing history. Each function is treated as a feature with its own FR.

2.3 User Characteristics

Primary users are casual players with basic computer literacy. Secondary users are evaluators or instructors reviewing the system.

2.4 Constraints

The program must use Java 17 or newer. It must operate offline, depend on CSV storage, and avoid any external network connectivity.

2.5 Assumptions and Dependencies

Users have valid file permissions and a Java runtime installed. The program depends only on standard Java libraries.

Note on Irrelevant Sections: IEEE subsections 2.6 and 2.7 are omitted because the software does not include external interfaces or specialized hardware.

3. Functional Requirements (Organized by Features)

The Functional Requirements (FRs) below are organized by features. Each FR includes inputs, processing, outputs, and an importance ranking (1=low, 2=medium, 3=high).

3.1 Feature – Game Board and Difficulty Levels

Importance Ranking: 3 (Reason: Essential game environment.)

3.1.1 Description

This FR defines the Game Board and Difficulty Levels feature. It interacts with the UI and internal logic to maintain gameplay consistency.

3.1.1.1 Inputs

User input events received through the UI during gameplay.

3.1.1.2 Processing

The Controller processes these inputs, updates the Model, and refreshes the UI View accordingly.

3.1.1.3 Outputs

Visual updates, messages, and data written to CSV files as required.

3.2 Feature – Player Management

Importance Ranking: 3 (Reason: Controls players and turns.)

3.2.1 Description

This FR defines the Player Management feature. It interacts with the UI and internal logic to maintain gameplay consistency.

3.2.1.1 Inputs

User input events received through the UI during gameplay.

3.2.1.2 Processing

The Controller processes these inputs, updates the Model, and refreshes the UI View accordingly.

3.2.1.3 Outputs

Visual updates, messages, and data written to CSV files as required.

3.3 Feature – Cell Interaction

Importance Ranking: 3 (Reason: Defines core interaction mechanics.)

3.3.1 Description

This FR defines the Cell Interaction feature. It interacts with the UI and internal logic to maintain gameplay consistency.

3.3.1.1 Inputs

User input events received through the UI during gameplay.

3.3.1.2 Processing

The Controller processes these inputs, updates the Model, and refreshes the UI View accordingly.

3.3.1.3 Outputs

Visual updates, messages, and data written to CSV files as required.

3.4 Feature – Quiz Questions

Importance Ranking: 2 (Reason: Adds cognitive challenge and diversity.)

3.4.1 Description

This FR defines the Quiz Questions feature. It interacts with the UI and internal logic to maintain gameplay consistency.

3.4.1.1 Inputs

User input events received through the UI during gameplay.

3.4.1.2 Processing

The Controller processes these inputs, updates the Model, and refreshes the UI View accordingly.

3.4.1.3 Outputs

Visual updates, messages, and data written to CSV files as required.

3.5 Feature – Scoring and Life System

Importance Ranking: 3 (Reason: Determines progress and fairness.)

3.5.1 Description

This FR defines the Scoring and Life System feature. It interacts with the UI and internal logic to maintain gameplay consistency.

3.5.1.1 Inputs

User input events received through the UI during gameplay.

3.5.1.2 Processing

The Controller processes these inputs, updates the Model, and refreshes the UI View accordingly.

3.5.1.3 Outputs

Visual updates, messages, and data written to CSV files as required.

3.6 Feature – Game End and Results

Importance Ranking: 3 (Reason: Handles conclusion and results.)

3.6.1 Description

This FR defines the Game End and Results feature. It interacts with the UI and internal logic to maintain gameplay consistency.

3.6.1.1 Inputs

User input events received through the UI during gameplay.

3.6.1.2 Processing

The Controller processes these inputs, updates the Model, and refreshes the UI View accordingly.

3.6.1.3 Outputs

Visual updates, messages, and data written to CSV files as required.

3.7 Feature – Question Management

Importance Ranking: 2 (Reason: Maintains the quiz repository.)

3.7.1 Description

This FR defines the Question Management feature. It interacts with the UI and internal logic to maintain gameplay consistency.

3.7.1.1 Inputs

User input events received through the UI during gameplay.

3.7.1.2 Processing

The Controller processes these inputs, updates the Model, and refreshes the UI View accordingly.

3.7.1.3 Outputs

Visual updates, messages, and data written to CSV files as required.

3.8 Feature – Game History

Importance Ranking: 2 (Reason: Allows reviewing past games.)

3.8.1 Description

This FR defines the Game History feature. It interacts with the UI and internal logic to maintain gameplay consistency.

3.8.1.1 Inputs

User input events received through the UI during gameplay.

3.8.1.2 Processing

The Controller processes these inputs, updates the Model, and refreshes the UI View accordingly.

3.8.1.3 Outputs

Visual updates, messages, and data written to CSV files as required.

3.9 Feature – Data Persistence

Importance Ranking: 3 (Reason: Ensures consistent data storage.)

3.9.1 Description

This FR defines the Data Persistence feature. It interacts with the UI and internal logic to maintain gameplay consistency.

3.9.1.1 Inputs

User input events received through the UI during gameplay.

3.9.1.2 Processing

The Controller processes these inputs, updates the Model, and refreshes the UI View accordingly.

3.9.1.3 Outputs

Visual updates, messages, and data written to CSV files as required.

4. Non-Functional Requirements

These requirements define system performance, usability, reliability, maintainability, and portability attributes.

4.1 Performance

All UI actions (revealing cells, marking flags) should respond within 200 ms. New games should load within one second.

4.2 Usability

The UI must be visually clear, intuitive, and responsive. It should use simple navigation and clear color schemes.

4.3 Reliability

The system should handle invalid input and missing files gracefully, automatically regenerating required CSVs.

4.4 Maintainability

The MVC structure ensures that modifications to the UI or logic can be done independently without major rework.

4.5 Portability

MiniSweeper must run identically across Windows, macOS, and Linux systems with a Java runtime environment.

4.6 Security

Input validation is required for all CSV operations. No user data is shared externally.

Note on Irrelevant Sections: Privacy laws and certification standards are omitted because MiniSweeper is an academic software project.

5. Appendices

5.1 Data Files

System still flexible, data files not ready yet.

5.2 Template Used

This document follows the Feature-Organized IEEE 830 SRS Template. It was chosen because it maps each system requirement to a visible feature, improving traceability and aligning naturally with Agile methodologies.

5.3 Notes on Irrelevant IEEE Sections

Hardware-related and legal sections were excluded. MiniSweeper is a self-contained application without hardware or communication dependencies.

*עבודה הייתה קבוצתית ולא מחולקת.